UNIVERSITY of VIRGINIA | DARDEN Business Publishing

UVA-OM-1822
Oct. 12, 2026

Vibe Coding for MBAs

The MBA who Codes

In the space of just a few years, building a greenfield 1.0 of a new application went from a months-long project to something you can do in an afternoon. Keeping that 1.0 online, supported, and improving is harder, and it has not gotten easier at anything like the same rate. An MBA who wants to bring new ideas to life, or to participate in digital executions at scale, needs to be able to do two things: understand the relationship between design and coding, and understand the foundational components of a digital infrastructure.

It's kind of like accounting. Even if it doesn't appeal to you, as a general manager (GM) you need to understand and engage with it, because it's a crucial part of operating the business. The same is now true of digital. And since AI continues to change just about everything about how software gets developed, this is a good time to engage more substantially in rethinking those processes with your colleagues and clients.

This note is about the 'how' of coding. It covers how an application comes apart into its component pieces, where software lives, how coding got to where it is, four repeatable steps that take a design to working code, and how to anchor all of it in version control. It assumes you already know how to state what you want built in a testable form, with user stories, overall evaluation criteria (OEC), and Given-When-Then scenarios.[1] Exhibit 2 has example inputs.

This note assumes certain things about you, the reader. It assumes that you're a student of business, familiar with accounting, finance, marketing, and the use of spreadsheets for analysis. It assumes that while you're enthusiastic about product development, your current trajectory is likely toward a bigger-picture role, something in GM or consultation within that domain. Given all that, this note will focus on the business context of coding and the core fundamentals of how it's done.

How Do You Unpack an Application with the MVC Framework?

Every industry has its lingo, but tech is notoriously complex. There are some good reasons for this- the diversity and intentionality of tech's various ecosystems is a large part of what made it so relatively inexpensive and productive, even pre-AI. No engineer knows every concept and every term, but just about all engineers

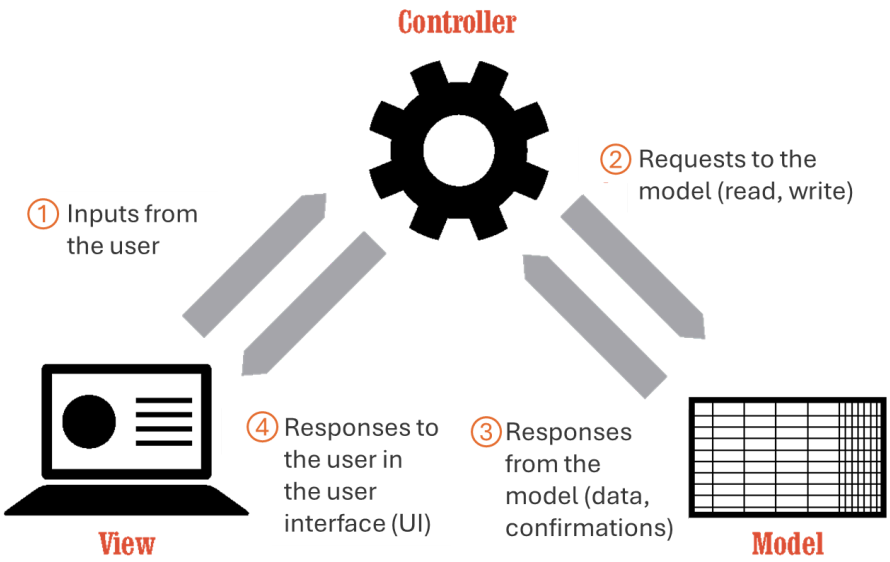
[1] For more on these topics, see Alex Cowan, "Storytelling with Vibe Code," UVA-OM-[number coming soon] (Charlottesville, VA: Darden Business Publishing, 2026).

This technical note was prepared by Alex Cowan, General Faculty, and Yael Grushka-Cockayne, Professor of Business Administration. Copyright © 2024 by the University of Virginia Darden School Foundation, Charlottesville, VA. All rights reserved. To order copies, send an email to sales@dardenbusinesspublishing.com. *No part of this publication may be reproduced, stored in a retrieval system, used in a spreadsheet, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—without the permission of the Darden School Foundation. This publication is protected by copyright and may not be uploaded in whole or part to any AI, large language model, or similar system, or to any related training database.* Our goal is to publish materials of the highest quality, so please submit any errata to editorial@dardenbusinesspublishing.com.

know how to know what's pertinent to the work in front of them. Basically, they know how to play animal-vegetable-mineral with tech terms to get to the understanding they need. You should too, and a good place to start is the model-view-controller (MVC) framework.

The basic idea with the MVC is to decompose a given application in terms of three foundational components: the application's model, view, and controller (see **Figure 1**). Basically, the (data) model is where the data lives, the view is what the user sees, and the controller is the logic that tells the application how it should respond to inputs from the user. The user passes inputs to the view, which then consults the controller, which usually needs to retrieve data from the model, do something else, and then return the updated data to the display in the view.

Figure 1. The MVC framework.



Source: Reproduced with permission from Alex Cowan, *Economics of Code: Advanced Product Management Series* (Cooke & McDouglas, 2023).

Let's apply this to something we do all day long: logging into an app. A user visits the site (or opens an app) and sees a view with some kind of affordance for entering their username and password. They type their username into the view, enter their password, and then press or tap a button to log in. Now the view passes those inputs to a controller, which executes a series of steps something like the following:

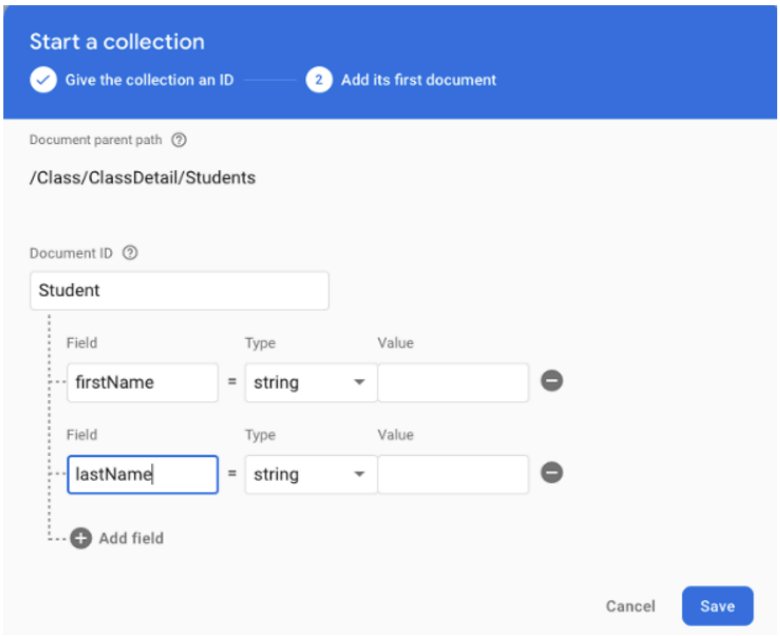
1. Check whether the username exists by looking it up in the model.
    a. If it does exist, proceed.
    b. If it doesn't exist, return an error to the view, letting the user know.
2. Check whether the password is correct by looking it up in the model.
    a. If it is correct, log the user in, fetch their recent purchases, and show those to them in the view.
    b. If it is not correct, return and send an error to the view, letting the user know.

To apply the MVC to the job of understanding your team's development, let's say you walk into the office and your colleagues in development are talking about TypeScript. Out of simple curiosity (a hallmark of good

PdMs), you ask ChatGPT what TypeScript is, and you learn that it's a superset of the JS programming language that some developers like because it's more particular about how one defines variables in code than regular JS. Programming languages, like JS, are generally for creating controllers—though a few, like structured query language (SQL, pronounced "sequel"), are more model oriented.

What else might that team be using to round out its MVC? Let's further suppose that the team is developing a web application—a customer relationship manager (CRM) for veterinarians. That would mean the team is using HTML and CSS to build views. How about its model? Well, that could be just about anything: It might use a relational database management system (RDBMS) like PostgreSQL/Postgres or an abstracted backend-as-a-service (BaaS) operation like Google Firebase's Firestore (**Figure 2**). RDBMS like Postgres or MySQL are actually their own stand-alone MVCs in that they have their own views (for configuration, looking at data, and so on), controllers (for reading and writing the data), and, of course, models (for how they structure the data).

Figure 2. Example user view for Google Firebase.



Source: Google Cloud Firestore. Google and the Google logo are trademarks of Google LLC, used with permission.

You will hear these three parts referred to by any number of other names, depending on who is talking and what they are building. The model gets called the database, the data structure, the data layer, or the back end. The view is the UI or the front end. The controller is the algorithm, the server-side code, or, confusingly, also the back end, since 'back end' tends to mean everything the user doesn't see. None of these are wrong, and you will hear all of them, sometimes in the same meeting.

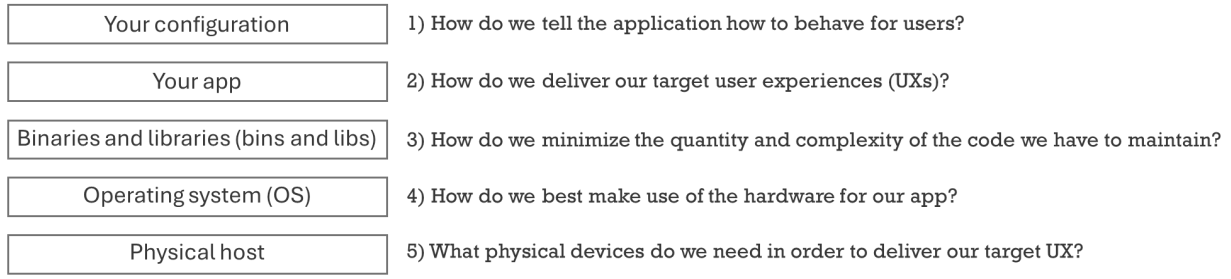
The next time you hear a tech term you don't know, try placing it: is it about the model, the view, or the controller? Then ask an AI or a colleague what job it does there. There's plenty of additional nuance, and plenty of terms that straddle two of the three, but this will get you to the right general ballpark to start asking more specific questions.

Where Does Software Live?

Code does not exist in isolation. You may have heard the one about how software is just '1's and 0's', but it is a tiny (and decreasing) proportion of engineers who work at that level. Most code sits on top of other code which sits on top of other code, etc. How exactly you 'stack' your code is equally, if not more, important than how you write it.

Layering is an integral concept in both design and engineering, and a tech stack is an example of layering. While there is no universal definition of how to layer the jobs of a software application in a "stack," the layering in **Figure 3** is pretty conventional.

Figure 3. Unpacking a stack.



Source: Created by author.

There's a configuration that tells the application how to behave, the app itself, bins and libs (prewritten third-party code you can leverage), the operating system (OS), and then the physical device (e.g., a server, an iPhone, or a PC). Let's take a closer look at these layers and how their jobs interact with various stack types you might encounter.

First, we have the job of configuring our application. Many applications are built to contain their own data that changes their behavior without having to change their underlying code. For example, feature flags are switches you can put into your code to dynamically turn certain features on or off for a given set of users—something you might do in, say, A/B testing. You may have heard of "codeless," "no-code," or "low-code" platforms like Airtable or ServiceNow—those are basically stacks that allow the app developer to operate with nothing more than a certain configuration of, for example, Airtable or ServiceNow.

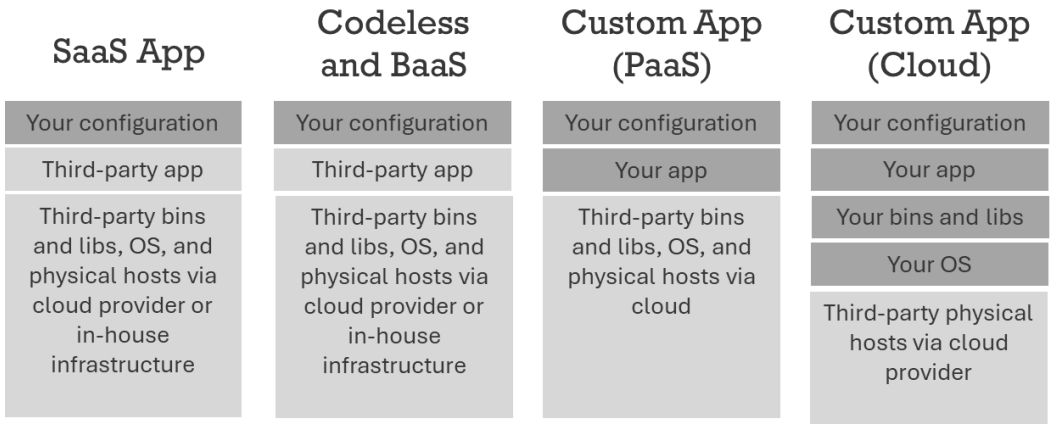
Next, you have the application itself: the custom code you write (your app) and the third-party building blocks (supporting code, usually open source) that help you do that (bins and libs). Next is the OS. This might be a mobile or PC OS, or, for cloud applications, an OS tuned for servers, like Linux or Windows Server. Finally, you have the physical hardware all this runs on.

This stack definition is a good way to contextualize the various environments you may encounter "under the hood" as you learn more about how your team operates. With a software-as-a-service (SaaS) application (e.g., Salesforce) or a codeless environment (e.g., Airtable or ServiceNow), you might be coding your application using various abstraction tools which help you configure an existing application in a way that delivers your target UX.

Within the conventional options for building your own application, you might use a platform-as-a-service (PaaS) (e.g., Heroku or Amazon Web Services [AWS] Amplify) where a third party handles the underlying stack (including bins and libs). Alternatively, you may be specifying the OS, bins and libs, and application if

you are building and operating a custom application in the cloud. Obviously, there's a non-cloud option as well, wherein you operate the underlying hardware (see **Figure 4**.)

Figure 4. Common stack choices.



Source: Reproduced with permission from Cowan, *Economics of Code*.

What Is Coding and When Is It Done?

If this question sounds too obvious, consider someone unfamiliar with business. If they asked you, for example, "Do you know how to do business?"; "Are you almost done with the business?"; or "How good are you at business?"; those questions would sound strange. Coding today is equally varied, and while the perception lingers that you code something and it's done, that notion is increasingly out of step with today's operating environment.

Asking a developer if they code well is the same as asking an MBA if they do business well: Developers probably have some idea about what "coding well" looks like, but they're always learning, and the applicability of their skills and experience will vary substantially based on the type of business they're doing.

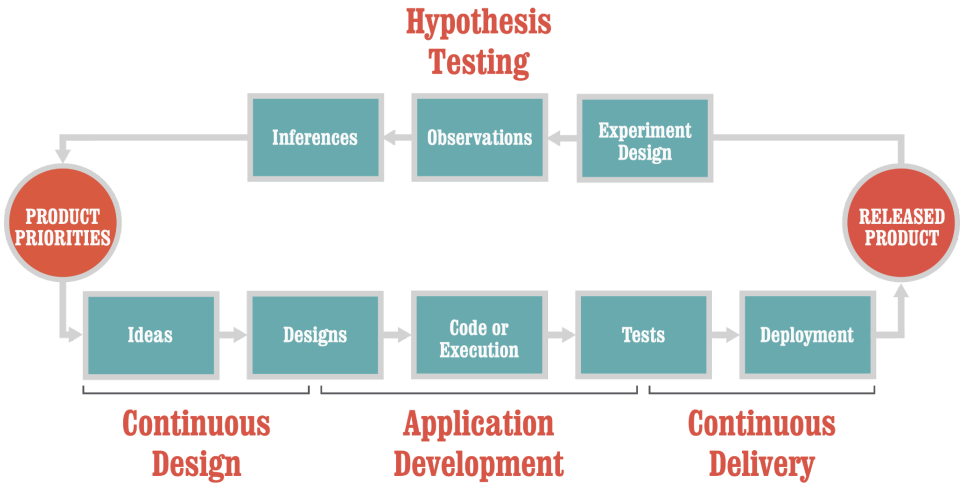
And what is it to learn to code? This is JavaScript (JS) code you can run in any web browser:

```
alert("Hello, World!");
```

It creates a pop-up that says "Hello, World!" It does this by executing a function called "alert" that posts text to the screen. Do you know how to code now?

From a GM perspective, one useful way to understand coding is as part of a digital product "pipeline" (**Figure 5**) where teams iterate from a certain set of priorities to designing, coding, testing, and deployment, and then they observe whether their ideas about how a user will respond to a design are right or wrong. Then the cycle repeats. The coding process takes designs as an input and then progresses them to testing and deployment to users. These days, various aspects of testing and deployment, and even design, often entail the use of code, but the process of going from application design to working code is likely to be the richest.

Figure 5. A product pipeline.



Source: Created by author.

How Did They Code Then?

How software gets built has always been driven a lot more by economics than most people realize. Tools and practices that look like matters of engineering taste usually turn out to be responses to whatever was scarce and expensive at the time. This matters to a manager for two reasons. A lot of what you used to need to know to work with code has changed, and mostly that means fewer details. At the same time, 'coding' has expanded into a much broader activity with a lot of subspecialties, and it helps to know how it got that way.

Three costs drivers explain most of the story: the cost of compute, the quality and depth of the supporting infrastructure a developer can build on, and the cost of development talent. Each has moved by orders of magnitude, and not always in the same direction.

In the first era, early computing, compute was the scarce thing and you couldn't do that many useful thing with it. The first Cray-1 supercomputer was installed at Los Alamos in 1976. It weighed around five tons, sold for around $8 million, and could do 160 million calculations a second, about 20 for every dollar it cost. By 2013, the chip in an ordinary smartphone did about six times that in a device that sold for a few hundred dollars, which is something like a 100,000 times gain in the cost of computation.[2]

There was hardly any supporting infrastructure, so teams wrote nearly everything from scratch, for one machine at a time. Talent, on the other hand, was relatively cheap, something like oil before there were a lot of cars and planes. You couldn't really do that much with a computer, and while the work was economically productive, it was nothing like today. It was closer to working in a biotech lab now: specialized, expensive to equip, and a long way from the day-to-day of most businesses.

The second era runs from the dot-com boom to the recent present. Compute stopped being the constraint, first as hardware got cheaper and then as cloud providers started renting it by the hour. Supporting infrastructure got deep. Open-source software went from a curiosity to the default, to the point where there is

[2] "Cray-1," Wikipedia, <https://en.wikipedia.org/wiki/Cray-1> (accessed Oct. 12, 2026). The first customer, the National Center for Atmospheric Research, paid $8.86 million in 1977, including disks.

now an open-source package for just about everything, meaning your typical developer deploys more than twice as much code that they didn't write as code that they did.[3]

Software started 'eating the world,' productivity jumped, and the scarce thing became the developer. Development talent was expensive and riding high, with pay that outpaced inflation: the median US software developer's salary rose about 10% in real terms between 2000 and 2022.[4] At elite shops like Google, the high end went much higher. By 2026, typical total compensation for a software engineer there ran from around $212,000 at the entry level to around $450,000 for a senior engineer.[5] Because talent was the bottleneck, a great deal was invested in tools and practices to get more from it: agile, extreme programming (XP), DevOps. All of that still pays off today. AI doesn't moot it, though it is certainly going to change it.

The third era is the present, and we're still in the relatively early days of understanding it. Compute is scarce again, but only on the AI frontier. For the ordinary work of building an application the cost of AI to an individual or organization is close to immaterial. Supporting infrastructure continues to be important and still a better alternative to custom code in most cases. The packages, platforms, and services of the second era are all still there, and there are more of them every year.

AI itself has become infrastructure: adaptive, humanistic automation for the work of developing software. The cost for the talent you need to harness it has become highly variable. The median software developer in the United States earned about $136,000 in 2025,[6] but there is much less need for junior and even mid-level developers. A Stanford study of US payroll data through September 2025 found that employment for software developers aged 22 to 25 fell nearly 20% from its late-2022 peak, while employment for developers aged 26 and older kept growing.[7] There will likely be fewer developers. What a typical coding job looks like in ten years, or for that matter a typical product manager's job, is still unclear.

For the MBA, the takeaway is that some durable foundational concepts are still clearly useful, and there are a few new ideas about how to put them to work. The next section covers both.

How Do You Code Now?

It's natural and intuitive to think of a coder as sitting down to work and constructing features like a carpenter builds the frame of a house. But it's not like that. First, all the simple, repetitive tasks of modern coding are already handled—a vast ecosystem of "helper code" (sometimes referred to as binaries and

[3] Black Duck's audit of commercial codebases found that 97% contained open source and that, on average, 70% of the scanned code had open-source origins. Black Duck, "Open Source Risk in 2025: What Developers Need to Know from the OSSRA Report," <https://www.blackduck.com/blog/qa-open-source-software-risk-2025.html> (accessed Oct. 12, 2026).

[4] Author's calculation. The US Bureau of Labor Statistics (BLS) reported a median annual wage of $67,670 for applications software engineers in 2000 and $127,260 for software developers in May 2022. The 2000 figure is about $115,000 in 2022 dollars, using annual average values of the Consumer Price Index (172.2 in 2000 and 292.7 in 2022). The BLS changed its occupational definitions between the two years, so the comparison is approximate. The 2000 BLS figures are as compiled in Nicole Kow, "The Evolution of Developer Salaries: Looking Back 20 Years," CodeSubmit, <https://www.codesubmit.io/blog/the-evolution-of-developer-salaries>; the 2022 figure is from BLS, "Occupational Employment and Wages, May 2022: 15-1252 Software Developers," <https://www.bls.gov/oes/2022/may/oes151252.htm>; CPI values are from Federal Reserve Bank of Minneapolis, "Consumer Price Index, 1913-," <https://www.minneapolisfed.org/about-us/monetary-policy/inflation-calculator/consumer-price-index-1913-> (all accessed Oct. 12, 2026).

[5] Self-reported total compensation in the United States, including base salary, stock, and bonus, for Google's L3 (entry-level) and L5 (senior) software engineers. Levels.fyi, "Google Software Engineer Salary," <https://www.levels.fyi/companies/google/salaries/software-engineer> (accessed Oct. 12, 2026).

[6] US Bureau of Labor Statistics, "Software Developers, Quality Assurance Analysts, and Testers," *Occupational Outlook Handbook*, <https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm> (accessed Oct. 12, 2026). The median annual wage for software developers was $135,980 in May 2025.

[7] Erik Brynjolfsson, Bharat Chandar, and Ruyu Chen, "Canaries in the Coal Mine? Six Facts about the Recent Employment Effects of Artificial Intelligence," Stanford Digital Economy Lab, November 13, 2025, <https://digitaleconomy.stanford.edu/app/uploads/2025/11/CanariesintheCoalMine_Nov25.pdf> (accessed Oct. 12, 2026).

libraries [bins and libs for short] or dependencies) already handles such tasks. How good is AI at using what already exists instead of writing more custom code? Left on its own, not very.

AI looks at custom code like a free puppy. New code costs an AI almost nothing to write, and the cost of owning that code lands later, on someone else. One analysis of 211 million changed lines of code found that as AI assistants spread, the share of changes that reorganized or reused existing code fell from 25% in 2021 to under 10% in 2024, while copied and pasted code rose from 8.3% to 12.3%.[8] An experienced developer weighs what code will cost to own. An AI has to be told to. Almost by definition, the code being written is "custom" to a particular purpose they've understood from you, the businessperson.

Second, and kind of by virtue of the first point, a lot of the job is identifying those component parts and making sure they work together properly. Here too, there are established patterns and available answers from generative AI, but this still takes up a substantial chunk of the time and effort a coder puts into a contemporary product or feature execution. Also, code gets complicated quickly, so most of the time, in practice, a coder's time is spent analytically testing and debugging code to ensure the right behaviors.

Finally, many if not most of the issues with code tie back to changing or ambiguous design intent. A classic failure mode for developers is to write a bunch of code, show it to the businessperson, and then find out it's not what the businessperson wanted. And what then? Even if you're developing code for your very own idea and design, new coders often stumble because they haven't adequately thought through the user experience (UX) they want to provide.

AI changes who does this work a lot more than it changes the work itself. AI can now be agentic from code through test and deployment, as long as it has a clear, testable intention to work from. The work still has to happen, though, and somebody still has to know whether it happened well.

Accounting for both the underlying economics of software development and the availability of AI as a cheap, enthusiastic helper, a reasonable way to organize your understanding of application development is around four principal steps:

1. Focusing and communicating design intent;
2. Unpacking what you want to have happen into codable steps;
3. Effectuating between alternatives; and
4. Testing and debugging.

These steps applied when people typed every line, and they apply now that AI does most of the typing. They also apply across the pipeline. The obvious case is the core application, but the same four steps hold for the infrastructure that tests it, deploys it, runs experiments on it, and secures it. What has changed is who does which part.

The example here is HVAC in a Hurry (HinH), a fictional regional heating, ventilation, and air conditioning (HVAC) service firm, and the application it is building to help field technicians order replacement parts.

We'll now take a look at the steps in more detail.

[8] GitClear, "AI Copilot Code Quality: 2025 Look Back at 12 Months of Data," 2025, <https://www.gitclear.com/ai_assistant_code_quality_2025_research> (accessed Oct. 12, 2026).

1. Focusing and communicating design intent

This is the step that stays with you. An AI that writes code in minutes will write the wrong code in minutes if the intention behind it is vague. A very detailed visual prototype in a design tool like Figma is a tempting way to say what you want ("make it look like this"), but it tells the AI, or a developer, relatively little about anything but the view/UI.

A better starting point is a testable user story, paired with a test prompt, overall evaluation criteria (OEC), and a few Given-When-Then scenarios. Take the case where Trent the Technician, one of HinH's field technicians, doesn't know the number of the part he needs:

As Trent the Technician,

I don't know the part number, and I want to try to identify it online

so I can figure out the next steps of the repair.

Its first Given-When-Then scenario might read:

Given Trent is signed in and knows the make and model of the unit he is repairing,

When he searches by make and model,

Then he sees the parts that fit that unit, each with its price and availability.

You can ask an AI to draft stories and scenarios, and you should read them the way you would read a draft from a new colleague who has never met your customer. The more specific and testable the intent, the less everything after it wanders.

2. Unpacking what you want to have happen into codable steps

Before code there is a plan. A common way to write one is "pseudocode," which is basically just a list of steps (or an algorithm). For Trent's story, pseudocode might look something like this:

1. Get the make and or model from the user.
2. Search for matching parts.
3. Show the user how many results you found and a description of each part.
    a. If there are no results, show the results as zero and return a descriptive notice like "no parts with parameters [description of the parameters the user supplied] found."

AI does this unpacking readily, so the job has changed from writing the plan to reading it. Ask to see the plan before the AI writes any code, because reading four steps is a lot cheaper than reading four hundred lines. If the plan surprises you, the problem is usually back in step 1.

The same unpacking applies to the other two parts of the MVC. A view unpacks into a set of nested rectangles, each with a job to do: a search form, a list of results, a single result. Developers call this the box model, and it is how a page gets laid out for anything from a phone to a desktop monitor.

A model unpacks into the real-world things the application keeps track of and how they relate, usually in terms of "has-a" and "is-a." A unit has a make and a model. A unit has parts. A compressor is a part. Getting these relationships right early matters, because everything else gets built on top of them.

Left on its own, AI will happily make all these assumptions for you. It may also fill in the blanks in a way that, in retrospect, was not the most intentional choice given what you know about your users and your business.

3. Effectuating between alternatives

In the first decade of the 2000s, it took a few million dollars to get a 1.0 release of a product out the door. Today, an equivalent application may cost only a few thousand dollars, or even less, to release. This is the result of a few major change catalysts, a big one being the availability of better building blocks, both commercial and open source.

Rather than starting from a blank sheet of paper, for today's coder, the choice of building blocks is a key determinant of their relative effectiveness in going from design to code. The coder could create the necessary functionality from scratch, adapt publicly available sample code from forums, use an open-source module, or leverage a third-party application programming interface (API)—or they could do a combination of those things. Good answers here are actually a combination of technical and economic considerations, so the less relevant a perspective you offer to your coder, the harder it is for them to make these decisions.

Let's say you want your application to send text messages to users—maybe for reminders, maybe for authentication, maybe both. You could add that capability with a third party API like Twilio—that's the easiest way, but it has the highest variable cost. You could implement it with open-source packages—that has a much higher fixed cost, but it's cheaper to send individual messages. Which one's the right answer? This is an area where a general manager's perspective on what may be to come and how that might affect the product's profit drivers may end up being important. However, without any intuitive sense of what it's like to go from design to code, it's harder to ask the right questions and engage on the answers.

The right answer depends a lot on where the thing in question sits relative to what makes the business distinctive. Consider three situations at HinH.

First, we'll consider their corporate website. HinH needs one, along with the marketing tools that attach to it, but none of that is HinH's area of distinction. A commercial off-the-shelf (COTS) site builder like Squarespace or Webflow does the job, and HinH would be crazy to build its own. AI makes custom code cheap to write. It does not make it cheap to own.

The second is the core internal application, the one Trent the Technician uses. HinH could build it on a platform like Salesforce. Instead, they build it custom because they believe that how HinH dispatches, diagnoses, and gets parts to a job site is its secret recipe, and that it will keep learning things it wants to render into the application. Here, custom code on a conventional, well-supported stack is a reasonable investment.

The third is the infrastructure for HinH's agentic UXs. HinH wants a lot of flexibility here, and it wants to steward its own proprietary data. That does not mean building every piece. Jobs like running AI evals or storing data at scale are their own disciplines with lots of short-term advances. Here, standard open-source or commercial components are the better bet, with HinH's own effort going into what it puts in them and how it assembles them.

Wrong choices tend to come in two flavors, and you can find both in each part of the MVC. On the view, a team hand-builds a custom interface for an internal tool that five people will use, when a standard component

library or off-the-shelf UI (Salesforce, etc.) would have done. Or it launches a consumer product on a no-code builder and then finds it can't run the experiments it needs. On the controller, a team writes its own login and payment handling, jobs that are easy to get almost right and expensive to get wrong. Or it wires its secret recipe into a vendor's proprietary workflow tool and finds it can't leave. On the model, a team stands up a sophisticated database cluster for a few thousand records. Or it keeps the business's most valuable data in a spreadsheet long after the application has outgrown it. In each pair, the first mistake is building what you should have bought, and the second is renting what you needed to own.

What these choices have in common is technical debt: short-term decisions that make subsequent changes more expensive. Like financial debt, it is not always bad. For a 1.0 whose job is to test an idea, taking some on is usually the right call, since most ideas do not survive and there is no point gold-plating a feature nobody uses. Once users, data, and money accumulate, the interest gets expensive. An AI will lay out alternatives like these and will have an opinion. What it does not know, unless you tell it, is where the business is headed. Good choices here combine technical and economic considerations, and the economic half is yours.

4. Testing and debugging

As natural as it is to think of coding like building a house or digging a trench, it just isn't. Given that:

A. most code we write today is new and novel;

B. because most basic or obvious functions are already available in various existing building blocks; and

C. that most code is actually just part of a module that furnishes a given end user application;

…getting everything to work together is tricky.

Rather than typing out lines of code like bricks in a foundation, a large part of a developer's time is spent answering questions like the following:

1. Do I understand well enough what this code is supposed to do for the user?
2. What are my inputs and my outputs?
3. What could go wrong and how do I plan for that?
4. Which approach to getting this done would be most consistent with our general approach to the code? Which would make it easiest for myself and my colleagues to debug and maintain?
5. What facilities do our existing building blocks (e.g., open-source components or licensed platforms that do things like manage authentication) offer for doing this?
6. Should I write custom code or can we instead use, entirely or in part, some already written code that someone else is maintaining?

With AI doing the coding, those questions are still the job, but they are now questions you put to the AI. The intent from step 1 says what to check, since each Given-When-Then scenario can be run as a test. When one fails, an analytical approach beats blind trial and error. Is the make and model arriving as entered? Is the search finding the parts? Are the results being displayed? Each question narrows where the problem can be. Asking the AI to run those checks and report which one fails gets better results than telling it to make the thing work.

The steps are a loop more than a line. A failed test often sends you back to the plan, and a muddled plan sends you back to the intent. That is normal, and it is the reason to keep each pass small.

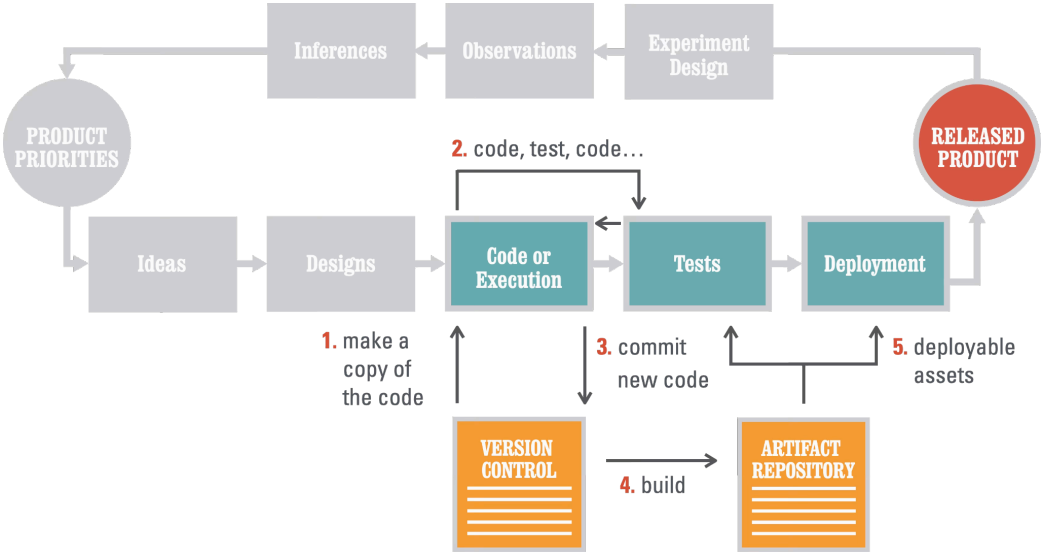
Where do you start?

There are a lot of tools for vibe coding, and more every month. Many of them, like Lovable, will take you from a prompt to a working application without your ever seeing a file. There is nothing wrong with that. For testing an idea, a code artifact that lives inside one of those tools may be all you need. But once you're operating at any kind of scale, running your work through a version control system (VCS) is the difference between a serious project and a side project.

The time to graduate to version control is when you're headed into serious deployment, and, as a new developer, there are good reasons for you to start sooner. It is the standard context for coding at scale, so working in it is the fastest way to understand how development teams operate. It also keeps your options open. As long as you have a well-organized repository (or 'repo'), you can collaborate with just about any programmer, and with just about any AI coding tool. The repo is the first thing either will probably ask for.

Figure 6 shows where version control sits in a product pipeline. A developer, or an AI working on the developer's behalf, starts by making a copy of the code (1). The developer then goes around the loop of coding and testing as many times as it takes (2), and commits the new code back to version control (3). From there, the committed code is built into deployable assets (4), which are kept in an artifact repository and are what actually gets deployed (5). Everything downstream of the developer starts from what is in version control. As far as the rest of the pipeline is concerned, a change that hasn't been committed doesn't exist.

Figure 6. Version control in the product pipeline.



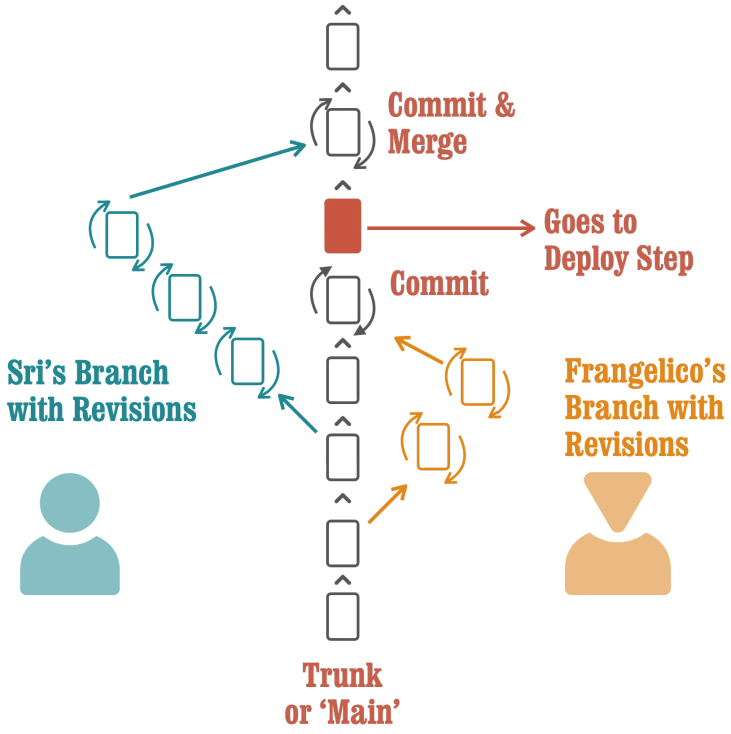
How does it work? Let's start with the difference between a document and a codebase. A document is one file, and if two people edit it at once, a tool like Google Docs merges their typing as they go. A codebase is many files that have to work together, where a change to one can break another, and where a team needs to know exactly what changed, who changed it, and why. A VCS handles this by recording changes as a series of

'commits.' Each commit is a snapshot of the whole codebase with a short message from its author, and any commit can be compared with any other or restored.

The dominant VCS is Git, and the most common place to keep a Git repo is GitHub. Getting a change from your computer to your team takes a few steps. You edit a file and save it. You stage the changes you want to include. You commit them, with a message that says what changed and why. Then you push the commit to GitHub, where your teammates can pull it down to their own copies. With an AI coding tool, you'll usually ask the AI to do the staging, committing, and pushing, but they are the same steps, and you should know which one you're on.

Figure 7 shows how this works with more than one person. The line up the middle is the trunk, usually called 'main,' and it is the version of the code that the team treats as current. Each box is a commit. Rather than committing straight to main, Sri and Frangelico each start a branch, which is their own line of commits that begins from main at the point where they started. They can revise on their branches without getting in each other's way. When a piece of work is ready, its author merges the branch back into main, which adds those commits to the trunk, and commits on main are what go on to the deploy step. A team that is just starting out can commit straight to main, as Exhibit 1 does, and take up branches once more than one person is changing the code at the same time.

Figure 7. Branches, commits, and merges on a shared trunk.



Keep commits small. A commit that does one thing is easy to describe, easy to review, and easy to undo if it turns out to be wrong. A commit that does ten things is none of those. This matters more with AI, which will cheerfully change twenty files when you asked about one. Small commits are how you keep track of what it did.

Exhibit 1 walks through the setup: creating a GitHub account, creating or joining a repo, making a first commit, and checking that it worked the way you expected.

Exhibit 1

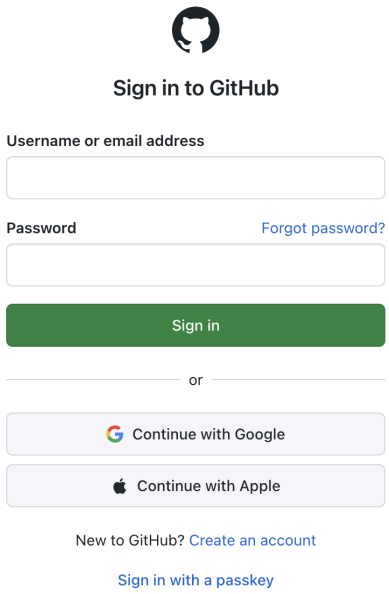
Vibe Coding for MBAs

Setting up Version Control

This exhibit walks a team through setting up version control and making its first commits. It sticks to the basics. The specifics of where to click change often, so for those, GitHub's own documentation (docs.github.com) is the place to look, and your AI coding tool will happily walk you through any step if you ask.

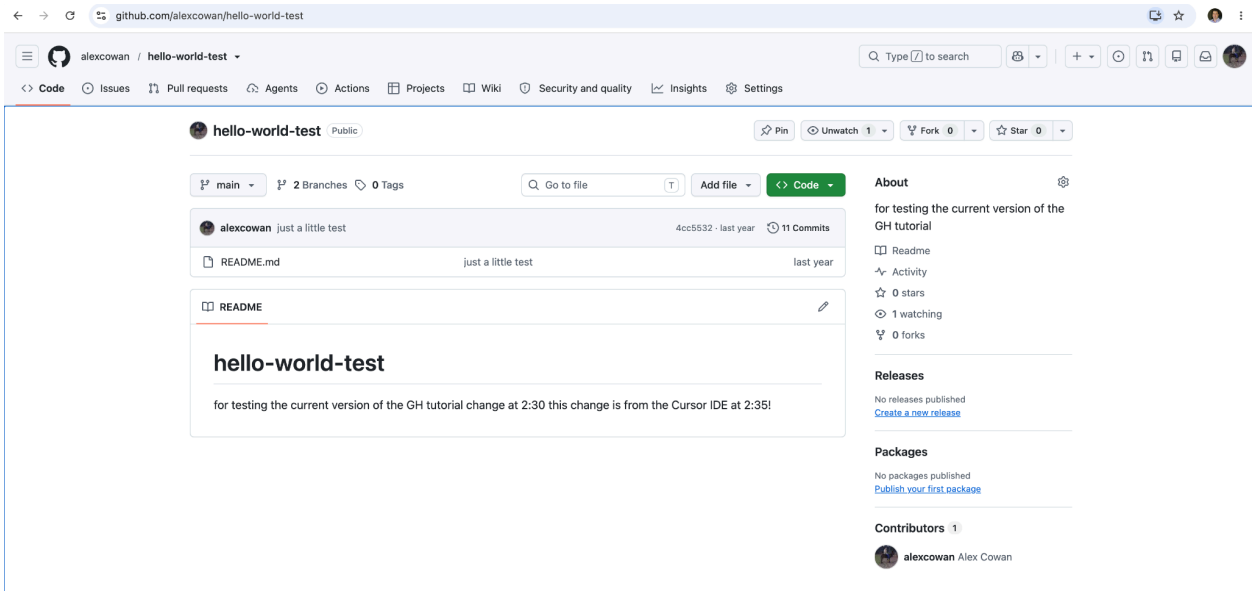
Getting set up

Find it. Git is the version control system. GitHub (github.com) is the most widely used service for hosting Git repositories, and it is where your team's repo will live.



Create an account. Each team member needs their own GitHub account, since the whole point is a record of who changed what. A free account is enough. Pick a username you wouldn't mind an employer seeing, and share your usernames with each other.

Create a repo. One team member creates a new repository for the project. Give it a short name with no spaces, make it private, and add a README file when prompted. A README is the front page of a repo, a plain-text description of what the project is, and adding one means the repo starts with something in it.



Invite collaborators. In the repo's settings, the team member who created it adds each teammate as a collaborator by username. Each teammate gets an invitation and has to accept it. Until they do, they can't push changes.

Clone it. Each team member makes a copy of the repo on their own computer, which is called cloning. Give your AI coding tool the repo's address and ask it to clone the repo, or use a free application like GitHub Desktop. You now have two copies that Git keeps in step: the one on your computer (local) and the one on GitHub (remote).

From save to commit, with a simple file

Do this first with a file that doesn't matter, so that the steps are familiar before the code is at stake. If you're working in an AI coding tool, you can ask it to do each step instead of typing the command. Have it tell you which step it is on, and check the result on GitHub yourself.

Create a file. In your copy of the repo, add a text file named for yourself (trent.txt, say), put one line in it, and save it. Saving changes the file on your computer and nothing else. Ask Git for its status (git status) and it will list your file as new and untracked.

Stage it. Tell Git that the file belongs in your next commit (git add trent.txt). Staging is how you choose what goes in and what waits, which matters once you have changed several files for different reasons.

Commit it. Record what you staged as a commit, with a short message on what changed and why (git commit -m "Add Trent's file"). The commit now exists on your computer, and only there.

Push it. Send your commit to GitHub (git push). Open the repo on GitHub and you should see your file, your message, and your username next to it.

Typed as commands, the four steps above look more or less like this. The lines that start with $ are what you type, and the rest is what Git says back.

```
$ git status
```

```
Untracked files:
        trent.txt
$ git add trent.txt
$ git commit -m "Add Trent's file"
[main 3f2a1c9] Add Trent's file
 1 file changed, 1 insertion(+)
$ git push
To https://github.com/hinh-team/parts-app.git
   8d41e07..3f2a1c9  main -> main
```

Update it. Change the line in your file, save, and go around again: status, stage, commit, push. This time, look at the file's history on GitHub. It shows both versions and what changed between them. This is the loop you'll repeat for as long as the project lives.

Do it with a peer. Start by pulling (git pull), which brings your teammates' commits down to your copy. Then add a line to a teammate's file, and stage, commit, and push. Have them pull and confirm they see your line. If two of you change the same line at the same time, Git will stop and report a conflict rather than guess whose change should win. Ask your AI coding tool to show you both versions, decide between them, and commit the result. Pulling before you start work and keeping commits small are how you keep conflicts rare.

Checklist

1. Every team member has their own GitHub account and has accepted the invitation to the team's repo.
2. Every team member has cloned the repo to their own computer.
3. The repo's history on GitHub shows at least one commit from each team member's own account.
4. Every team member has pulled a change that someone else made and can see it in their own copy.
5. The README says, in a sentence or two, what the project is and who is on the team.

Key operations

Operation	What it does	Command
Clone	**Makes your own copy of the team's repo on your computer. You do this once.**	**git clone [address]**
Pull	**Brings your teammates' commits down to your copy. Do it before you start work.**	**git pull**
Save	**Changes a file on your computer. Git notices, but nothing is recorded yet.**	**(your editor)**
Status	**Lists what has changed and what is staged. A good first question to ask Git.**	**git status**
Stage	**Picks which changes go into the next commit, and which wait.**	**git add [file]**

Commit	**Records a snapshot of what you staged, with a message on what changed and why. It exists only on your computer.**	**git commit -m "[message]"**
Push	**Sends your commits to GitHub, where your teammates can pull them. Until you push, no one else can see them.**	**git push**

Exhibit 2

Vibe Coding for MBAs

Example Design Inputs for HVAC in a Hurry's Parts Application

These are the inputs that the four steps start from for the example in this note. Each is specific and testable, and any of them could turn out to be wrong.

User story. As Trent the Technician, I don't know the part number, and I want to try to identify it online so I can figure out the next steps of the repair.

Test prompt. Hand the subject a part where only the make and type are visible and ask how they would look it up. Did they reach the part's pricing and availability?

Overall evaluation criteria (OEC). How well does this search work relative to the alternatives? The metrics are searches of this type relative to others, and conversion to order from this type of search. Customer satisfaction per job and billable hours must not fall for the technicians who use the tool.

Given-When-Then scenarios. Given Trent is signed in and knows the make and model of the unit he is repairing, when he searches by make and model, then he sees the parts that fit that unit, each with its price and availability. Given Trent enters a make and model that match no parts in the catalog, when he searches, then he sees that there are no results for what he entered, and he is offered a way to get help identifying the part.